

1. Características básicas del lenguaje

JavaScript se distingue de otros lenguajes de programación por un conjunto de características que definen su comportamiento y su forma de ejecutarse. Comprenderlas es el primer paso para entender por qué el lenguaje se comporta como lo hace.

1.1 Lenguaje interpretado

A diferencia de lenguajes compilados como C o Java, JavaScript no se traduce por completo a código máquina antes de ejecutarse. En su lugar, un motor de JavaScript (como V8 en Chrome y Node.js, o SpiderMonkey en Firefox) lee el código y lo ejecuta línea por línea en tiempo real. Los motores modernos utilizan compilación JIT (Just-In-Time), que traduce el código a instrucciones optimizadas mientras se ejecuta, logrando un rendimiento cercano al de los lenguajes compilados sin perder la flexibilidad de un lenguaje interpretado.

1.2 Lenguaje dinámico

JavaScript utiliza tipado dinámico: una misma variable puede almacenar valores de distinto tipo a lo largo de la ejecución del programa, y el tipo se determina en tiempo de ejecución, no al declarar la variable.

```
let dato = 42;           // dato es Number
dato = "hola";          // ahora dato es String
dato = true;            // ahora dato es Boolean
```

1.3 Lenguaje multiparadigma

JavaScript no obliga a seguir un único estilo de programación. Soporta:

- **Programación imperativa:** instrucciones secuenciales que modifican el estado del programa paso a paso.

- **Programación orientada a objetos (basada en prototipos):** a diferencia de Java o C++, JavaScript no usa clases reales internamente (aunque la sintaxis "class" las simula); los objetos heredan comportamiento directamente de otros objetos a través de la cadena de prototipos.
- **Programación funcional:** las funciones son "ciudadanas de primera clase", es decir, se pueden guardar en variables, pasar como argumentos y retornar desde otras funciones.

1.4 Otras características relevantes

- **Débilmente tipado:** JavaScript convierte tipos de datos automáticamente en muchas operaciones (coerción de tipos), por ejemplo "5" + 3 da como resultado "53".

- **Basado en eventos:** gran parte de su uso en el navegador depende de reaccionar a eventos del usuario (clics, teclas, envíos de formularios).
- **Single-threaded con event loop:** JavaScript ejecuta una sola tarea a la vez en un solo hilo, pero gracias al event loop puede simular concurrencia manejando tareas asíncronas sin bloquear la ejecución principal.

2. Estructuras de control

Las estructuras de control determinan el orden en que se ejecutan las instrucciones de un programa. Permiten tomar decisiones y repetir bloques de código, lo cual es esencial para construir lógica compleja.

2.1 Condicionales

Los condicionales evalúan una expresión booleana y ejecutan un bloque de código distinto dependiendo de si el resultado es verdadero o falso.

```
if (edad >= 18) {
  console.log("Es mayor de edad");
} else if (edad >= 13) {
  console.log("Es adolescente");
} else {
  console.log("Es menor de edad");
}

// switch: útil cuando hay muchos casos posibles para un mismo valor
switch (diaSemana) {
  case 6:
  case 0:
    console.log("Fin de semana");
    break;
  default:
    console.log("Día laboral");
}

// operador ternario: forma compacta de un if/else
const mensaje = edad >= 18 ? "Adulto" : "Menor";
```

2.2 Bucles

Los bucles permiten repetir un bloque de código mientras se cumpla una condición, evitando escribir instrucciones repetidas manualmente.

```
// for: cuando se conoce el número de iteraciones
for (let i = 0; i < 5; i++) {
  console.log(i);
}

// while: se repite mientras la condición sea verdadera
let contador = 0;
while (contador < 3) {
  contador++;
}

// do...while: se ejecuta al menos una vez
do {
  contador--;
} while (contador > 0);

// for...of: recorre valores de un iterable (arrays, strings)
for (const fruta of ["manzana", "pera"]) { console.log(fruta); }

// for...in: recorre las claves (propiedades) de un objeto
for (const clave in { nombre: "Ana", edad: 20 }) { console.log(clave); }
```

3. Tipos de datos

JavaScript clasifica sus datos en dos grandes categorías: tipos primitivos, que se almacenan y comparan por valor, y tipos por referencia, que se almacenan y comparan por referencia en memoria.

3.1 Tipos primitivos

- **String:** cadenas de texto, por ejemplo "Hola mundo".
- **Number:** representa tanto enteros como decimales, por ejemplo 42 o 3.14.
- **Boolean:** valores lógicos true o false.

Hecho por: Carrillo Ruiz Angel Sebastian Información sobre Java Scrip

Materia: Programación Web

- **Undefined:** una variable declarada pero sin valor asignado.
- **Null:** representa la ausencia intencional de un valor.
- **BigInt:** para números enteros de precisión arbitraria, mayores al límite seguro de Number.
- **Symbol:** valores únicos e inmutables, usados frecuentemente como identificadores de propiedades.

```
typeof "texto";      // "string"  
typeof 10;           // "number"  
typeof true;         // "boolean"  
typeof undefined;    // "undefined"  
typeof null;         // "object" (peculiaridad histórica del lenguaje)
```

3.2 Tipos por referencia

Los objetos, arrays y funciones son tipos por referencia: cuando se asignan a otra variable, no se copia el valor, sino la dirección de memoria donde se encuentra el dato.

```
const obj1 = { valor: 1 };  
const obj2 = obj1;      // obj2 apunta al mismo objeto en memoria  
obj2.valor = 99;  
console.log(obj1.valor); // 99, porque ambas variables referencian lo mismo
```

4. Funciones

Las funciones son bloques de código reutilizables diseñados para realizar una tarea específica. JavaScript ofrece tres formas principales de definir las, cada una con matices importantes.

4.1 Funciones declaradas

Se definen con la palabra clave `function` y un nombre. Se benefician del hoisting, es decir, pueden invocarse antes de su definición en el código porque el motor las registra al inicio del ámbito.

```
sumar(2, 3); // funciona aunque se llame antes de la definición
```

```
function sumar(a, b) {  
  return a + b;  
}
```

4.2 Funciones expresadas

Se asignan a una variable. No tienen hoisting de su cuerpo, por lo que no pueden llamarse antes de la línea donde se definen.

```
const restar = function (a, b) {  
  return a - b;  
};
```

4.3 Funciones flecha (arrow functions)

Introducidas en ES6, ofrecen una sintaxis más corta y no poseen su propio valor de `this`, sino que heredan el del contexto donde fueron definidas (esto es clave al trabajar con objetos y clases).

```
const multiplicar = (a, b) => a * b;  
  
// con múltiples líneas se requieren llaves y return explícito  
const dividir = (a, b) => {  
  if (b === 0) return null;  
  return a / b;  
};
```

5. Objetos y arrays

5.1 Objetos

Hecho por: Carrillo Ruiz Angel Sebastian Información sobre Java Script

Materia: Programación Web

Un objeto es una colección de pares clave-valor. Es la estructura fundamental para representar entidades del mundo real dentro del código.

```
const persona = {  
  nombre: "Sebas",  
  edad: 21,  
  saludar() {  
    return `Hola, soy ${this.nombre}`;  
  },  
};  
  
console.log(persona.nombre);    // notación de punto  
console.log(persona["edad"]);   // notación de corchetes
```

5.2 Arrays

Un array es una lista ordenada de elementos, indexada desde 0, que puede contener cualquier tipo de dato, incluyendo otros arrays u objetos.

```
const numeros = [10, 20, 30, 40];  
  
const dobles = numeros.map(n => n * 2);    // [20, 40, 60, 80]  
const pares = numeros.filter(n => n % 20 === 0); // [20, 40]  
const suma = numeros.reduce((acc, n) => acc + n, 0); // 100
```

- **map()**: transforma cada elemento y devuelve un nuevo array del mismo tamaño.
- **filter()**: devuelve un nuevo array solo con los elementos que cumplen una condición.
- **reduce()**: combina todos los elementos en un único valor acumulado.

6. Eventos y manipulación del DOM

El DOM (Document Object Model) es la representación en forma de árbol que el navegador crea a partir del HTML de una página. JavaScript puede leer y modificar este árbol dinámicamente, lo que permite crear interfaces interactivas sin recargar la página.

6.1 Selección de elementos

```
const boton = document.querySelector("#miBoton");  
const items = document.querySelectorAll(".item");  
const titulo = document.getElementById("titulo");
```

6.2 Manipulación del DOM

```
titulo.textContent = "Nuevo título";  
titulo.style.color = "blue";  
titulo.classList.add("resaltado");
```

6.3 Eventos

Los eventos permiten ejecutar código en respuesta a acciones del usuario o del navegador, como clics, pulsaciones de teclas o el envío de un formulario.

```
boton.addEventListener("click", () => {  
  console.log("El botón fue presionado");  
});  
  
document.addEventListener("keydown", (evento) => {  
  console.log(`Tecla presionada: ${evento.key}`);  
});
```

7. Asincronía

La asincronía permite que JavaScript ejecute tareas que tardan tiempo (como peticiones a un servidor) sin detener el resto del programa mientras espera el resultado. Es uno de los

Hecho por: Carrillo Ruiz Angel Sebastian Información sobre Java Scrip

Materia: Programación Web

temas más importantes del lenguaje porque casi toda interacción con el mundo exterior (APIs, archivos, temporizadores) es asíncrona.

7.1 Callbacks

Un callback es una función que se pasa como argumento a otra función, para ser ejecutada una vez que cierta tarea haya finalizado.

```
setTimeout(() => {  
  console.log("Esto se ejecuta después de 2 segundos");  
}, 2000);
```

7.2 Promesas (Promise)

Una promesa representa un valor que estará disponible en el futuro. Puede encontrarse en tres estados: pendiente, resuelta (fulfilled) o rechazada (rejected).

```
const obtenerDatos = new Promise((resolve, reject) => {  
  const exito = true;  
  if (exito) resolve("Datos obtenidos");  
  else reject("Ocurrió un error");  
});  
  
obtenerDatos  
  .then((resultado) => console.log(resultado))  
  .catch((error) => console.error(error));
```

7.3 Async/await

Es una sintaxis introducida en ES2017 que permite escribir código asíncrono con una apariencia sincrónica, mejorando notablemente la legibilidad frente al uso de `.then()` encadenado.


```
    async function cargarUsuario() {  
  try {  
    const respuesta = await fetch("https://api.ejemplo.com/usuario");  
    const datos = await respuesta.json();  
    console.log(datos);  
  } catch (error) {  
    console.error("Error al cargar el usuario:", error);  
  }  
}
```